***SecureSend* Security Design Document:**

Ashiyana Ahamed - CS 419 Computer Security - Professor Kushwaha

SecureSend takes our project and reflects a real-world situation that emphasizes the importance of being able to securely send documents. In the legal field, particularly in practices which contain confidential information protected by the healthcare field's privacy protection guidelines through HIPAA (Health Insurance Portability and Accountability Act), transmitting medical records and similar patient protected information in a secure manner is incredibly important. Not only is this vital in the context of ensuring compliance with regulations, but also out of a principle to protect patient and client information. This project aims to send information and simulate a potential HIPAA compliant document sharing website, while this project is not fully HIPAA compliant (in details mentioned below as well), the project utilizes important principles that are a part of creating HIPAA compliant software. We document our success and limitations as well.

During penetration testing, all major security controls passed. Three low-severity findings were identified and remediated: expired session accumulation, HTML error pages exposing server information, and the Express framework fingerprint via the x-powered-by header.

A. Architecture Overview (10 points)

a. System architecture diagram

```
[Browser - React App]
        |
        | HTTPS (TLS)
        |
[Express Server - port 3001]
        |
    [Middleware]
    Security Headers → CORS → JSON → Rate Limiter
        |
    [Routes]
    /api/login  /api/register  /api/documents  /api/admin
        |
    [File Storage]
    users.json  sessions.json  documents.json
    encrypted_docs/  security.log  access.log
```

SecureSend follows a two tier architecture with a client and server tier. The frontend reflects the "client tier" while the backend reflects the "server tier".

Frontend (Client)

- React single-page application built with Vite
- Runs on http://localhost:5173 in development
- Communicates with the backend exclusively via HTTPS REST API calls
- JWT token stored in localStorage and attached to every authenticated request via Authorization header

Backend (Server)

- Node.js with Express framework
- Runs on https://localhost:3001 with TLS enabled via self-signed RSA 2048-bit certificate
- Handles all business logic, authentication, authorization, and encryption
- File-based storage using JSON files -> no database

`data/users.json` — user accounts with bcrypt-hashed passwords
`data/sessions.json` — active session tokens with expiry timestamps
`data/documents.json` — document metadata (filenames, owners, sharing permissions)
`data/encrypted_docs/` — AES-256 encrypted document files stored as `.enc` files
`logs/security.log` — comprehensive security event log
`logs/access.log` — Protected Health Information (PHI) access audit trail

b. Data flow diagrams

<u>Registration Flow:</u>

User fills form → Frontend validates (regex, complexity, match)
↓
POST /api/register over HTTPS
↓
Backend validates all fields (whitelist approach, independent of frontend)
↓
Check duplicate username/email in users.json
↓
bcrypt.hashSync(password, 12); hash with cost factor 12
↓
Write user to users.json: { id, username, email, hash, role: "user", ... }
↓

Log REGISTER_SUCCESS to security.log
↓
Return { success: true } → Frontend shows success screen

Login Flow:

User submits credentials → POST /api/login over HTTPS
↓
Rate limiter: >10 requests/min from this IP? → 429 Too Many Requests
↓
Find user in users.json
↓
Account locked? (lockedUntil > now) → 403 Account locked
↓
bcrypt.compareSync(password, storedHash)
↓
Failure: increment failedAttempts
→ ≥5 failures: set lockedUntil = now + 15min, log ACCOUNT_LOCKED
→ <5 failures: log LOGIN_FAILED attempt N/5
↓
Success: reset failedAttempts, create JWT (30min expiry)
↓
Save session to sessions.json with expiresAt timestamp
↓
Log LOGIN_SUCCESS → Return { token, user: { username, role } }
↓
Frontend stores token in localStorage, navigates to dashboard

Document Upload:

User selects PDF/DOCX → POST /api/documents/upload over HTTPS
↓
requireAuth: verify JWT signature and expiry
↓
Check role !== "guest" → 403 if guest
↓
multer fileFilter: validate extension AND MIME type
→ Invalid: return 400 "Only PDF and DOCX files are allowed"
↓
safeName(): path.basename() strips path traversal, regex validates chars
↓
Generate UUID for document ID and encrypted filename
↓

encryptFile(buffer):
  → Generate 16-byte random IV
  → AES-256-CBC encrypt buffer with IV + ENCRYPTION_KEY
  → Write [IV + encrypted data] to data/encrypted_docs/{uuid}.enc
  ↓
Save metadata to documents.json (encryptedPath never sent to frontend)
  ↓
Log DOC_UPLOAD to security.log AND access.log
  ↓
Return { success: true, id, name }

Document Download:
User clicks Download → GET /api/documents/:id/download over HTTPS
  ↓
requireAuth: verify JWT
  ↓
Find document in documents.json
  ↓
Check: isOwner || isSharedWith(user)
  → Neither: log ACCESS_DENIED, return 403
  ↓
Read encrypted .enc file from disk
  ↓
decryptFile():
  → Extract first 16 bytes as IV
  → AES-256-CBC decrypt remaining bytes
  ↓
Log DOC_DOWNLOAD to security.log AND access.log
  ↓
Send decrypted bytes as attachment
  ↓
Frontend creates blob URL, triggers browser download dialog

Share Flow:
Owner clicks Share → enters username/email + role
  ↓
POST /api/documents/:id/share over HTTPS
  ↓
requireAuth: verify JWT
  ↓
Verify requesting user is document owner → 403 if not
  ↓

```
Look up target user in users.json by username or email
  → Not found: return 404
      ↓
Add/update { username, role } in document's sharedWith array
      ↓
Write updated documents.json
      ↓
Log DOC_SHARE to security.log AND access.log
      ↓
Frontend reloads document list
```

c. Component descriptions

Navbar Component: Renders different navigation options based on user role. Admin users see an additional "Admin" link. This is a frontend UX control; all real authorization enforcement happens on the backend.

LoginPage Component: Handles user authentication. Sends credentials to the backend over HTTPS. On success, stores the JWT token in localStorage and updates application state.

RegisterPage Component: Handles user registration with full client-side validation matching backend requirements. Displays a real-time password strength indicator.

DashboardPage Component: Main document management interface. Loads documents from the backend on mount. Supports upload (with drag-and-drop), download, share, and delete operations. Guest users see a restricted view with no upload or share capabilities.

AdminPage Component: Available only to admin-role users. Displays real user data and security logs fetched from the backend. Supports locking/unlocking user accounts and exporting the full audit log.

Express Backend: Single `app.js` file containing all middleware, route handlers, encryption utilities, and the security logger. Middleware runs in order: CORS → JSON parsing → Security headers → Route handlers.

d. Technology stack justification

For the frontend framework, the project utilizes React + Vite which is a component based architecture which allows for a bit more cleaner coding as opposed to writing a giant HTML file. This way we are able to separate parts of the web application such as NavBar, LoginPage & DashboardPage which are all able to exist as separate components of a larger structure. Vite was used in tandem with React in order to update only the parts of the app that was edited rather than reloading everything which is what I would previously do without Vite, it only processes what is needed at that point. Also, due to prior experience with React, this was an easier choice.

The backend is built with Node.js and Express, as recommended by the project specification for real-time features. Node.js allows the same language to be used across both frontend and backend, reducing context switching. Express gives full control over middleware ordering, which was key in implementing the security pipeline (CORS → JSON parsing → security headers → rate limiting → authentication).

For password hashing, bcrypt was utilized as recommended by the project specification which is the industry standard for password encryption. The session tokens were JWT (json web token) & the 30 minute expiry limits exposure from stolen tokens. The file encryption is a built in Node.js module using AES-256 Cipher Block Chaining (CBC) which is NIST approved and meets the HIPAA requirements we aim to reach with this project. The file upload is handled by multer which is a widely used and well maintained middleware. Ratelimiting is handled by express-rate-limit npm package which is a simple way to handle brute force attacks. Transport security is handled by a self-signed TLS certificate, the API traffic is encrypted between frontend and backend. For the purposes of this project, self-signed made sense but if this were to be actually deployed, a certficate from a Certficate Authority would be required. Unique identifiers were also used through uuid v4 which uses cryptography to prevent ID ennumeration attacks. Axios was used as an HTTP client which also simplified error handling and header management.

B. Threat Model (10 points):

a. Asset identification

i. Medical Documents (in our example this is what we are classifying the documents as) -> Patient Health Information (PHI) where unauthorized disclosure can cause legal and regulatory harm as well as ethical concerns.

ii. User Passwords -> compromises in access to user passwords can enable accounts to be taken over and allow for unauthorized PHI access.

iii. JWT tokens -> Stolen tokens would allow for impersonation for up to 30 minutes
iv. User PII -> usernames, email addresses and account metadata, this is all personal information that should be protected
v. Audit logs -> Tampering with logs could result in compliance issues and concerns with incident response inaccuracy
vi. Encryption Keys -> compromise of this could result in decryption of all stored information, highly critical to protect.

b. Threat enumeration (using STRIDE model)

i. Spoofing
    1. Threat: Attacker impersonates a legitimate user
    2. Controls: bcrypt hashing, JWT authentication, account lockout, rate limiting
ii. Tampering
    1. Threat: Attacker modifies documents in transit or at rest
    2. Controls: TLS prevents in-transit modification. AES-256 encryption protects at-rest data. Files replaced on re-upload, not patched in place.
iii. Repudiation
    1. Threat: User denies performing an action on PHI
    2. Controls: Comprehensive append-only audit logging with timestamps and usernames. HIPAA audit log exportable.
iv. Information Disclosure
    1. Threat: Unauthorized access to medical documents or user data
    2. Controls: RBAC on every API route. Encrypted file paths never exposed to frontend. Generic error messages. x-powered-by header removed.
v. Denial of Service
    1. Threat: Automated flooding of login endpoint
    2. Controls: Rate limiter blocks >10 requests/min per IP. Account lockout after 5 failures.
vi. Elevation of Privilege
    1. Threat: User or guest attempts to access admin functionality
    2. Controls: requireAdmin middleware on all admin routes. Role verified server-side on every request. JWT signature prevents role tampering.

c. Vulnerability assessment

Vulnerability	Present?	Mitigation
Weak password storage	No	bcrypt cost 12, passwords never stored in plaintext
Brute force login	Mitigated	Rate limiting + account lockout after 5 attempts
Session hijacking	Partially	30-min JWT expiry + TLS -> localStorage storage noted as informational finding
Cross-site scripting (XSS)	Mitigated	React auto-escaping + CSP header + input regex validation
SQL injection	Not applicable	No SQL database -> JSON file storage used
Path traversal	Mitigated	safeName() strips directory components before saving
Command injection	Mitigated	No shell commands used -> Node.js file APIs only
Privilege escalation	Mitigated	requireAuth + requireAdmin on every protected route
Insecure file uploads	Mitigated	Extension + MIME type whitelist, memory storage before encryption
Information disclosure	Mitigated	Generic error messages, x-powered-by disabled, custom 404 handler
Unencrypted data at rest	No	AES-256-CBC encryption before any file touches disk
Unencrypted transport	No	TLS on all API traffic
Hardcoded secrets	Partially	Environment variables with dev fallback -> if in production, it should use env vars

d. Attack scenarios

Scenario 1: Brute Force Password Attack Automated login attempts to guess user passwords.

Mitigations:

- Rate limiter: max 10 attempts/min per IP → 429 response
- Account lockout: 5 failures → 15-minute lockout
- bcrypt cost 12: ~250ms per comparison makes large-scale attacks impractical
- All failures logged at WARNING severity

Scenario 2: Path Traversal Attack Upload file with malicious name like `../../etc/passwd.pdf` to write outside intended directory.

Mitigations:

- `path.basename()` strips all directory components
- Regex `^[\w\-\.]+$` rejects path separators
- Files stored in memory by multer before validation — never touch disk unencrypted

Scenario 3: Privilege Escalation Guest or user attempts to access admin-only routes or another user's documents.

Mitigations:

- requireAuth verifies JWT signature on every protected route
- requireAdmin checks role for admin routes
- Document access checks owner and sharedWith arrays before serving
- Tested and verified during penetration testing

Scenario 4: XSS Attack Injecting malicious JavaScript through input fields.

Mitigations:

- React auto-escapes all rendered content
- CSP header restricts script sources to 'self'
- Username regex rejects HTML special characters

Scenario 5: Man-in-the-Middle Attack Intercepting API traffic to steal JWT tokens or document contents.

Mitigations:

- All API traffic encrypted with TLS
- 30-minute token expiry limits exposure window
- HTTPS enforced at server level

e. Risk prioritization

Risk	Likelihood	Impact	Priority	Status
Stolen JWT token	Medium	High	High	Mitigated by 30-min expiry + TLS
Brute force	High	High	High	Mitigated by rate limit + lockout
Path traversal	Low	Critical	High	Mitigated by safeName()
XSS	Medium	High	Medium	Mitigated by React + CSP
Encryption key exposure	Low	Critical	High	Use env vars in production
Information disclosure via errors	Low	Low	Low	Remediated through custom error handlers

C. Security Controls (10 points):

USER AUTHENTICATION:

Control Description: Username/password authentication with bcrypt hashing, account lockout, and rate limiting.

Implementation:

- Username: 3-20 characters, alphanumeric and underscore only (`/^[a-zA-Z0-9_]{3,20}$/`)
- Email: format validation (`/^[^\s@]+@[^\s@]+\.[^\s@]+$/`)
- Password: min 12 characters, uppercase, lowercase, number, special character, enforced both client-side and server-side independently
- bcrypt cost factor 12 -> `bcrypt.hashSync(password, 12)`
- Plaintext passwords never stored or logged
- Account locked 15 minutes after 5 consecutive failures

- Rate limiting: max 10 login requests/min per IP via express-rate-limit
- Sessions saved to `sessions.json` with `createdAt` and `expiresAt` timestamps
- Generic "Invalid credentials" error -> does not reveal whether username exists (prevents enumeration)

Testing Results: PASS -> account locked after 5 attempts, rate limiter triggered at request 11, password complexity enforced on frontend and backend independently.

Known Limitations:

- No multi-factor authentication
- No password reset flow (identified as future enhancement)

ACCESS CONTROL:

Control Description: Role-Based Access Control with three roles representing real-world users in the law firm / medical records use case. Enforced at the API layer on every request via middleware, independent of the frontend.

Admin -> represents the law firm's IT administrator. Has full system access, including the ability to view ALL documents across all users, manage user accounts (lock/unlock), and export the HIPAA audit log. There is one admin account created by registering normally and manually setting `"role": "admin"` in `users.json`.

User (lawyer role) -> represents lawyers at the firm. Can upload, download, share, and delete their own documents. Can download documents shared with them. Cannot see other users' documents unless explicitly shared.

Guest (doctor role) -> represents physicians receiving medical records. Read-only access to documents explicitly shared with them. Cannot upload, share, delete, or access any admin functionality. This implements the principle of minimum necessary access so doctors only see what they need to see.

Permission Matrix:

Feature	Admin	User (Lawyer)	Guest (Doctor)
Upload documents	✓	✓	✗
Download own documents	✓	✓	✗

Download shared documents	✓	✓	✓
Share documents	✓	✓	✗
Delete own documents	✓	✓	✗
View all documents	✓	✗	✗
Manage users	✓	✗	✗
Export audit logs	✓	✗	✗

Enforcement:

Frontend (UX only): Admin link, upload button, share and delete buttons hidden based on role. Not a security control.

Backend (real security):

- `requireAuth` -> verifies JWT on every protected route
- `requireAdmin` -> checks role for admin-only routes, returns 403 if not admin
- Document routes verify ownership (`doc.owner === req.user.username`) and shared access (`doc.sharedWith.some(...)`) before serving
- Guest upload explicitly blocked with role check returning 403

Testing Results: PASS -> all four authorization tests blocked correctly and logged to security.log.

Known Limitations:

- Editor role stored in metadata but no in-place editing; requires re-upload
- Admin role assigned manually in users.json; no role management UI

INPUT VALIDATION:

Control Description: Whitelist-based validation on all user inputs, file uploads, and API parameters.

- Username: strict regex whitelist, 3-20 character limit
- Email: format validation regex
- Password: complexity requirements enforced independently on both client and server

- File uploads: extension whitelist (`.pdf`, `.docx`) AND MIME type validation (`application/pdf`, `application/vnd.openxmlformats-officedocument.wordprocessingml.document`)
- Filename sanitization: `path.basename()` + alphanumeric regex prevents path traversal
- React auto-escaping prevents XSS in all rendered content
- Content Security Policy restricts script execution to 'self'
- No raw SQL queries; JSON file storage eliminates SQL injection surface entirely
- Custom error handler returns JSON for all errors including multer rejections

Testing Results: PASS -> XSS payload rejected, path traversal neutralized, command injection rejected, malicious file upload rejected with clean 400 JSON response.

ENCRYPTION:

Control Description: AES-256-CBC at rest for documents; TLS for transport; bcrypt for passwords.

Data at Rest:

- Algorithm: AES-256-CBC
- Key: derived via `crypto.scryptSync()` from configurable master key (environment variable `ENCRYPTION_KEY` with dev fallback)
- IV: 16 cryptographically random bytes per file via `crypto.randomBytes(16)`
- Storage format: `[16-byte IV][encrypted data]` in UUID-named `.enc` files
- Encrypted filenames have no relation to original filenames
- Unencrypted data never written to disk, multer uses `memoryStorage()`

Transport:

- TLS via self-signed RSA 2048-bit certificate generated with OpenSSL
- All API communication over HTTPS (port 3001)
- Frontend configured to call `https://` endpoints
- Self-signed certificate causes browser warning

Testing Results: PASS -> encrypted .enc files verified unreadable, download decrypts correctly, all requests confirmed over HTTPS in browser Network tab.

Known Limitations:

- Self-signed certificate causes browser warning
- Encryption key uses env variable with hardcoded dev fallback

SESSION MANAGEMENT:

Control Description: JWT-based sessions with 30-minute expiry, persisted to sessions.json, cleaned up on logout.

Implementation:

- JWT created on login: `jwt.sign({ id, username, role }, JWT_SECRET, { expiresIn: "30m" })`
- Token sent in `Authorization: Bearer <token>` header on every authenticated request
- `requireAuth` verifies signature and expiry on every request
- Session record saved to `sessions.json` with `createdAt` and `expiresAt`
- On logout: current session token removed AND all expired sessions purged (remediated during pen testing)
- Token removed from localStorage on logout

Testing Results: PASS -> token cleared on logout, 30-minute expiry confirmed from JWT payload, expired sessions cleaned up on logout (low finding remediated).

Low Finding (Remediated): Expired sessions were accumulating in sessions.json across multiple login sessions. Fixed by adding expiry filter to logout route.

SECURITY HEADERS:

Control Description: HTTP security headers applied to every response via Express middleware, placed before CORS to ensure all responses include them.

Headers Implemented:

X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, XSS filter, Referrer-Policy, Permissions-Policy, Content-Security-Policy

Additional hardening:

- `app.disable("x-powered-by")` -> removes Express fingerprint header (remediated during pen testing)
- Headers set manually in middleware -> no third-party library dependency

Note: Security headers middleware is placed before CORS middleware to ensure headers are set on all responses, including preflight responses.

Testing Results: PASS -> all six headers confirmed present in browser Network tab response headers. x-powered-by header absent after remediation.

LOGGING & MONITORING:

Control Description: Structured JSON logging of all security-relevant events, append-only, with separate PHI audit trail.

Log Format (every entry):

```
json

{
  "timestamp": "2026-04-29T00:27:54.283Z",
  "severity": "INFO|WARNING|ERROR",
  "event": "EVENT_TYPE",
  "username": "actor_username",
  "detail": "human readable description"
}
```

Events logged to security.log:

- `LOGIN_SUCCESS`, `LOGIN_FAILED`, `LOGIN_BLOCKED`
- `REGISTER_SUCCESS`, `REGISTER_FAILED`
- `ACCOUNT_LOCKED`, `ACCOUNT_UNLOCKED`
- `LOGOUT`
- `ACCESS_DENIED`
- `INVALID_TOKEN`
- `DOC_UPLOAD`, `DOC_DOWNLOAD`, `DOC_SHARE`, `DOC_DELETE`
- `DOC_UPLOAD_FAILED`, `DOC_DOWNLOAD_FAILED`
- `LOG_EXPORT`

Events additionally logged to access.log (HIPAA PHI audit trail):

- `DOC_UPLOAD`, `DOC_DOWNLOAD`, `DOC_SHARE`, `DOC_DELETE`

Implementation details:

- `fs.appendFileSync` -> append-only, never overwrites

- Admin HIPAA export: `GET /api/logs/export` downloads full security.log as `.txt` file
- Export event itself logged to security.log

Testing Results: PASS -> all tested events appeared in security.log with correct fields, PHI access events duplicated in access.log, export download confirmed working.

Known Limitations:

- Logs stored on same server -> production should use remote log aggregation
- No log integrity verification (checksums, WORM storage)

D. Data Protection (10 points):

a. Data classification

Data Type	Classification	Storage	Protection
Medical documents	Restricted (PHI)	Encrypted .enc files	AES-256-CBC at rest, TLS in transit
Passwords	Confidential	users.json	bcrypt hash only, never plaintext
JWT tokens	Confidential	Client localStorage	HTTPS transmission, 30-min expiry
User metadata	Internal	users.json	Access controlled
Audit logs	Internal	logs/ directory	Append-only, admin-only export
Session data	Internal	sessions.json	JWT signature prevents tampering

b. Encryption methods

AES-256-CBC (Documents at rest)

- Key size: 256 bits (32 bytes)
- Mode: CBC (Cipher Block Chaining)

- IV: 128-bit random, unique per file -> prevents pattern analysis
- Key derivation: `crypto.scryptSync()` with configurable password via environment variable
- Implementation: Node.js built-in `crypto` module

bcrypt (Passwords)

- Algorithm: bcrypt
- Cost factor: 12 (2^12 = 4096 iterations per comparison)
- Salt: automatically generated per password
- Implementation: bcryptjs library

TLS (Transport)

- Certificate: Self-signed, RSA 2048-bit, generated with OpenSSL
- Scope: All API traffic between React frontend and Express backend

c. Key management

- Encryption key derived at server startup via `crypto.scryptSync()`
- Configurable via `ENCRYPTION_KEY` environment variable
- Falls back to development default if not set -> production MUST use env var
- Key never written to disk -> exists only in server memory during runtime
- JWT secret similarly configurable via `JWT_SECRET` environment variable

d. Secure deletion procedures

When a document is deleted:

E. Encrypted `.enc` file removed from disk via `fs.unlinkSync()`
F. Document metadata entry removed from `documents.json`
G. Deletion logged to both `security.log` and `access.log`

HIPAA COMPLIANCE CONSIDERATIONS:

SecureSend was designed with HIPAA Technical Safeguards in mind for the use case of law firms sharing medical records with physicians.

Technical Safeguards Implemented

HIPAA Requirement	Implementation
Unique user identification	UUID per user, unique username enforced
Emergency access procedure	Admin can unlock any account
Automatic logoff	JWT expires after 30 minutes
Encryption/decryption	AES-256-CBC at rest, TLS in transit
Audit controls	security.log and access.log with full event history
Integrity controls	Encrypted files detect tampering
Transmission security	TLS encrypts all API communication
PHI access audit export	Admin downloads full audit log for compliance reporting

Gaps in HIPAA for future work

- Email verification -> emails stored but not verified for ownership
- Business Associate Agreements — legal contracts, not a technical implementation
- Data backup -> production requires automated encrypted backups of data/ and logs/
- Breach notification -> documented procedures required for production
- Remote log aggregation -> logs should be shipped offsite in production

Known Limitations:

- Self-signed TLS certificate causes browser warning -> production would need CA-signed cert
- Encryption key has hardcoded dev fallback -> production must use `ENCRYPTION_KEY` env var
- No email verification on registration
- No password reset flow
- No document versioning -> re-upload replaces file
- Logs stored on same server -> production should use remote aggregation

Future Enhancements:

- CA-signed TLS certificate (Let's Encrypt)

- Email verification and password reset
- Multi-factor authentication
- Document versioning with full history